

0321 선대

왜 머신러닝의 기초가 선형대수인가?

1. 세상에 있는 것들을 연산할 수 있다
2. 정보의 본질을 정의할 수 있다

Preliminaries

Vector

Feature

Embeddings

선형대수

Basis

Linear Transformation

Dot Product

Rank

Eigenvalues & Eigenvectors

ML 워밍업: 평가지표

모델을 평가하는 기준

Confusion Matrix

Accuracy

Precision

Recall

F1 Score

왜 머신러닝의 기초가 선형대수인가?

1. 세상에 있는 것들을 연산할 수 있다

현실의 데이터는 컴퓨터로 연산 불가능하다

사과, 바나나, 사람의 목소리 같은 현실의 데이터는 컴퓨터가 바로 이해할 수 없다

따라서 사과와 바나나가 같은 과일이라는 것도 컴퓨터는 알 수 없다

컴퓨터가 이해할 수 있는 것은 숫자이다

→ 선형대수는 현실세계를 숫자의 공간으로 옮겨준다

사과, 바나나, 사람의 목소리를 **Vector**로 바꾸면,
사과와 바나나가 얼마나 비슷한지 계산할 수 있다

2. 정보의 본질을 정의할 수 있다

우리가 보는 데이터에는 **유용한 정보**와 **노이즈**가 섞여있다

선형대수는 데이터의 수많은 방향중 '어느 방향이 진짜 중요한 정보는 담고 있는 축인지' 찾아낸다

ex) 개와 고양이를 구분할 때 수만 개의 픽셀 데이터 중에서
귀의 모양이나 눈의 위치 같은 **결정적인 basis** 축을 찾아낸다

Preliminaries

Vector

현실의 데이터가 숫자의 공간으로 들어오는 순간 우리는 이것 벡터라고 함

1. 텍스트 → 벡터

단어와 문장은 고유한 의미를 가진 숫자의 나열로 변함

ex) "사과"라는 단어를 [0.5, -0.2, 0.1, ...] 같은 수백 차원의 좌표로 변환

비슷한 의미를 가진 단어들은 공간상에서 **서로 가까운 위치**에 찍히게 됨

2. 이미지 → 벡터

이미지의 각 픽셀이 가진 밝기(0~255)나 색상 정보를 일렬로 쭉 세우면 하나의 긴 벡터가 됨

고양이 사진 벡터와 강아지 사진 벡터를 비교하여, **어떤 숫자의 패턴**이 다른지 계산으로 찾아냄

3. 로봇 상태 → 벡터

로봇이나 자율주행 차가 지금 어디에 있고 어떤 상태인지도 벡터로 표현됨

[현재 위치, 속도, 관절의 각도(proprioception)] 등을 하나의 임베딩으로 묶음

이 벡터를 입력받아 다음에 취해야 할 행동 벡터(action chunk)를 계산해냄

머신러닝 모델은 Vector를 입력받아 유용한 정보를 남긴 새로운 Vector를 출력한다

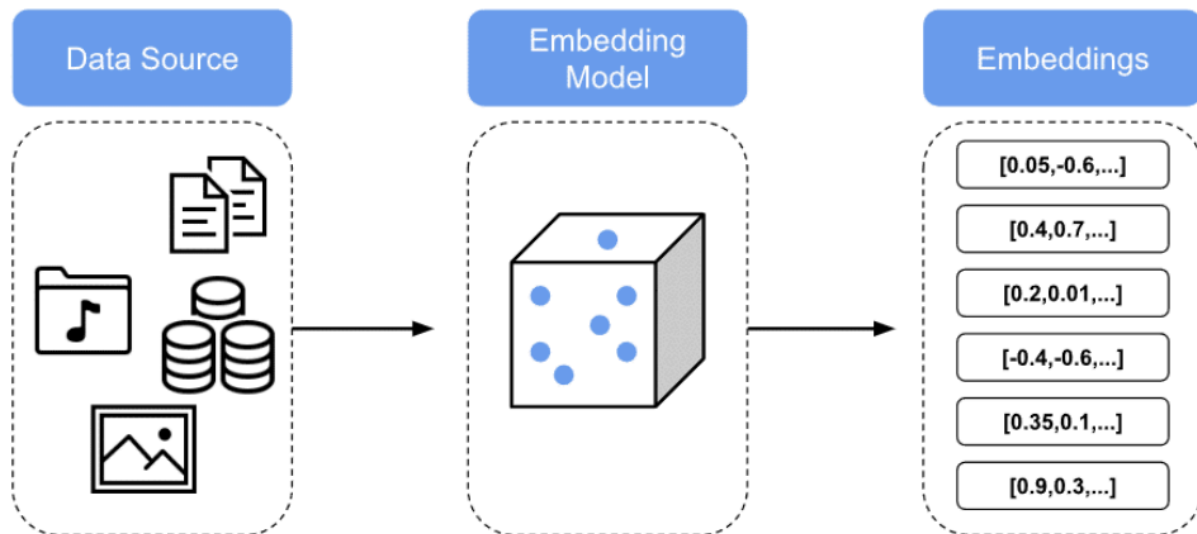
Feature

: 벡터의 각 차원

- Latent feature : 딥러닝이 스스로 찾아낸 Feature. 사람은 직관적으로 이해하기 어려움
- Feature space : feature들이 축이 되어 형성되는 공간 (=vector space)

Embeddings

임베딩은 의미를 가진 Vector



Vector와 Embedding의 차이

‘어떤 방식으로 생성되었는가’와 ‘어떤 정보를 가지고 있냐’로 구분할 수 있음

- Vector
 - 데이터에서 무엇을 볼지 **사람이 미리 정해놓은 가이드라인**에 따라 숫자를 뽑아낸 것
 - 기준이 많아질수록 0이 많아지고 데이터가 들성들성해짐
 - 분위기나 미묘한 맛의 차이처럼 말로 설명하기 어려운 특징은 놓치기 쉬움
- Embedding
 - 데이터를 해석한 벡터. 데이터 사이의 관계와 맥락을 보존하면서 컴퓨터가 이해하기 가장 좋은 형태로 압축한 것
 - 데이터를 신경망에 넣어 AI가 판단해 압축한 숫자들
 - 숫자 하나하나가 정확히 무엇을 의미하는지 사람은 알기 어려움
 - 적은 차원 안에 정보가 짹짹 눌러 담겨있음
 - 데이터 사이에 숨겨진 미묘한 관계를 잘 찾아냄

왜 임베딩을 하는가?

1. 비슷한 것은 가깝게

‘왕’과 ‘왕비’라는 단어는 텍스트로는 다르지만 임베딩 공간에서는 매우 가까운 곳에 위치함

2. 관계를 계산

임베딩 공간에서는 왕-남자+여자 = 여왕 같은 벡터 연산이 가능해짐

⇒ 임베딩은 데이터의 Noise(쓸데없는 축)들을 버리고,
데이터의 본질을 가장 잘 설명하는 축(Basis)로 이루어진 Vector로 변환하는 것

선형대수

Basis

| 조건

벡터 공간 V 의 모든 벡터가 $v_1, v_2, \dots, v_n$ 의 선형결합으로 표현되고

이들이 서로 선형 독립일 때

| basis란

vector space의 모든 곳을 가리키기 위해 필요한 최소한의 축들

- 한 vector space의 basis는 무수히 많이 존재 가능
- 같은 data라도 어떤 basis를 기준으로 보느냐에 따라 값이 완전히 달라짐

| ML에서의 basis : 유의미한 축을 발견

ML&DL에서 모델이 학습한다는 것 = 데이터를 가장 효율적으로 설명하는 새로운 basis를 찾는 것

ex1. PCA (주성분 분석)

- 데이터의 분산이 가장 큰 방향을 첫번째 축으로 삼음
- 수많은 Feature 중 데이터의 차이를 가장 잘 설명하는 축을 찾아 차원을 축소하는 과정

ex2. Word Embedding

- 수만 개의 단어를 나열하는게 아닌, 수백 개의 basis를 가진 공간에 배치함
- 학습이 잘 된 모델 = basis가 잘 뽑힌 모델

Linear Transformation

| 분류하기 좋도록 공간을 왜곡시키는 것

데이터를 더 넓은 공간으로 펼치거나 특정 방향으로 모음

→ 특징을 더 잘 잡아내는 새로운 data

| 선형변환 만으로는 문제를 풀기 어려움

비선형성이 필요함 → non linear activation에 대해서도 배우게 될 것

Dot Product

두 벡터가 유사한지 다른지 판단하는데 주로 쓰임

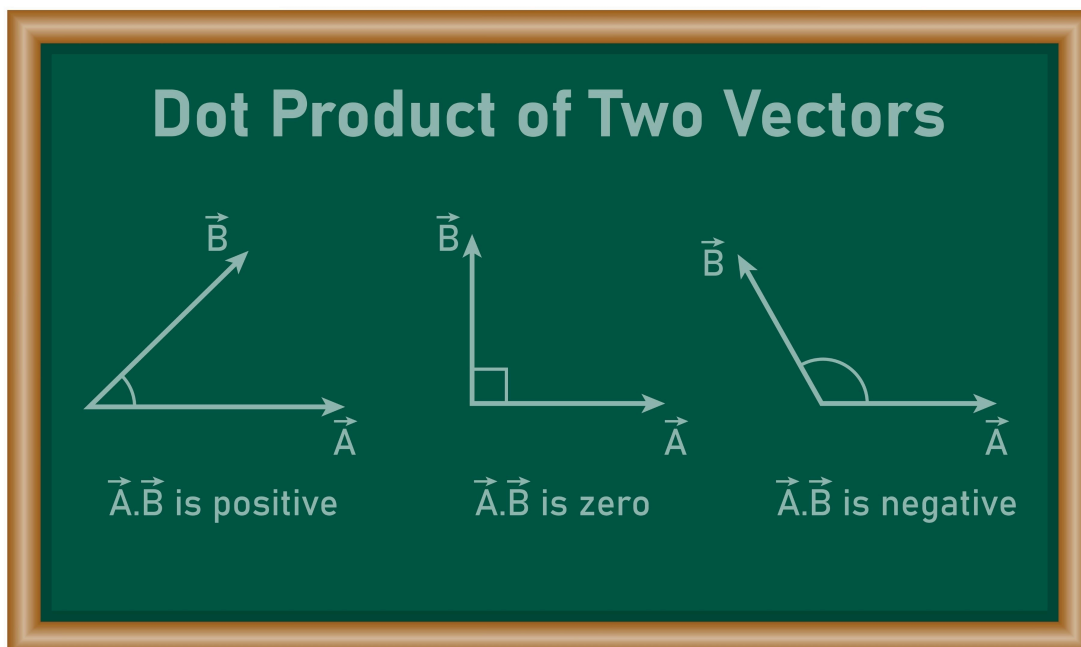
$$a \cdot b = \sum a_i b_i = \|a\| \|b\| \cos \theta$$

사과, 바나나, 사람의 목소리가 있다고 할 때

인간은 사과와 바나나가 같은 과일이라는걸 알지만 컴퓨터는 모른다

→ 내적을 통해 단어들의 의미가 유사한지 판단하도록 함

| 내적 값



>0 두 벡터가 같은 방향 : 의미가 비슷함

=0 : 상관관계가 없음

<0 두 벡터가 다른 방향 : 의미가 다름

| ML&DL에서 어떻게 쓰이나

일종의 filtering 역할을 함

벡터 b 를 기준으로, 벡터 a 를 input이라고 생각해보면,

a 에 수많은 정보가 섞여 있어도 b 와 내적을 하는 순간 b 와 전혀 상관없는 정보는 0이 되어 날아감

→ b 의 방향과 일치하는 애들만 살릴 수도 있음

| 왜 내적을 쓰는가

유사도를 측정하는 방법은 유클리드 거리 등 많음.

내적을 쓰는 이유는 연산속도가 빠르기 때문

내적 = 곱하고 더하는 것의 반복 → GPU가 제일 잘하는 일

Rank

| Rank = 정보의 진짜 농도

100×100 행렬이라고 해서 100가지 특징을 담고 있는 건 아님

- linear dependence) 어떤 열이 다른 열들의 조합으로 만들어질 수 있다면, 그 열은 새로운 정보를 제공하지 않음 → 중요한 열이 아님
- Rank : 새로운 정보를 제공하는 독립적인 axis의 개수

| Full Rank vs Low Rank

- Full Rank : 모든 축이 제각각 다른 정보를 담고 있어 버릴 게 없는 상태
- Low Rank : 축은 많은데 몇 개의 핵심 정보가 반복되거나 섞인 상태

| real world data는 대개 Low Rank

고해상도 고양이 사진은 수백만 개의 픽셀(차원)로 이루어져 있음

→ 픽셀 하나하나가 완전히 독립적인 정보를 가지진 않음

왼쪽 눈 픽셀이 검은색이면 바로 옆 픽셀도 검은색일 확률이 매우 높음

→ 수백만 차원의 공간에 점이 찍혀 있지만, 실제 고양이 데이터들이 모여 있는 진짜 차원 (Rank)은 훨씬 낮음

⇒ 모델이 학습한다는 것 = Intrinsic Rank를 찾는 과정

Eigenvalues & Eigenvectors

| 방향이 안바뀜

보통 행렬 A 를 벡터 x 에 곱하면(Ax)

벡터는 방향도 바뀌고 길이도 변함

- eigenvector : 고정된 방향
- eigenvalue : 그 방향으로 얼마나 stretch 됐는지 나타

| ML : 주성분 찾는데 쓰임

- **eigenvector = 데이터의 흐름 방향**: 이 데이터들이 가장 길게 늘어서 있는 방향이 어디인가
- **eigenvalue = 그 방향의 중요도**: 그 방향으로 데이터가 얼마나 많이 퍼져 있는가? (분산)

→ eigenvalue가 클수록 그 방향에 정보가 많이 몰려있다는 것

ML 워밍업: 평가지표

모델을 평가하는 기준

1. Evaluation Metrics → 오늘 설명할 것

- 모델 자체의 실력을 평가
- 모델이 수치적으로 얼마나 정확하게 예측하는가?

2. Benchmarks

- 내 모델 vs 다른 모델(baseline)
- ex) MMLU, ImageNet 등
- 다른 모델들과 비교할 때 내 모델이 수준이 어느정도인지를 확인하는 목적

Confusion Matrix

모델이 예측한 결과와 실제 정답이 얼마나 '혼되고 있는지 보여주는 표'라고 해서 Confusion Matrix..

		예측 클래스 (Predicted Class)	
		Negative(0)	Positive(1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

- **TP (True Positive)**: Positive로 예측해서 맞춤 (정답)
- **TN (True Negative)**: Negative로 예측해서 맞춤 (정답)
- **FP (False Positive)**: Positive로 예측했는데 틀림 (오답, **Type I Error**)
- **FN (False Negative)**: Negative로 예측했는데 틀림 (오답, **Type II Error**)

Accuracy

$$\frac{TP + TN}{TP + TN + FP + FN}$$

- 가장 기본적인 지표.
- Accuracy = 전체 데이터 중 맞게 예측한 비율
- 데이터 불균형이 심할 때 (ex. 암환자가 1%인 경우)는 신뢰하기 어려움

Precision

$$\frac{TP}{TP + FP}$$

- Precision = 모델이 Positive라고 예측한 것 중 실제로 Positive인 비율
- ex) 스팸메일 분류 : 스팸이 아닌데 스팸으로 분류하면 중요한 메일을 못읽으므로 이런 task에서 중요한 지표

Recall

$$\frac{TP}{TP + FN}$$

- Recall = 실제 Positive인 데이터 중 모델이 Positive라고 찾아낸 비율
- ex) 암 진단 : 암환자를 정상으로 분류하면 위험하니 이런 task에서 중요한 지표

F1 Score

$$2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- Precision과 Recall은 Tradeoff 관계이므로 이를 합친 지표
- F1 Score = Precision과 Recall 중 어느 하나로 치우치지 않았나